



# **Predavanje 4**

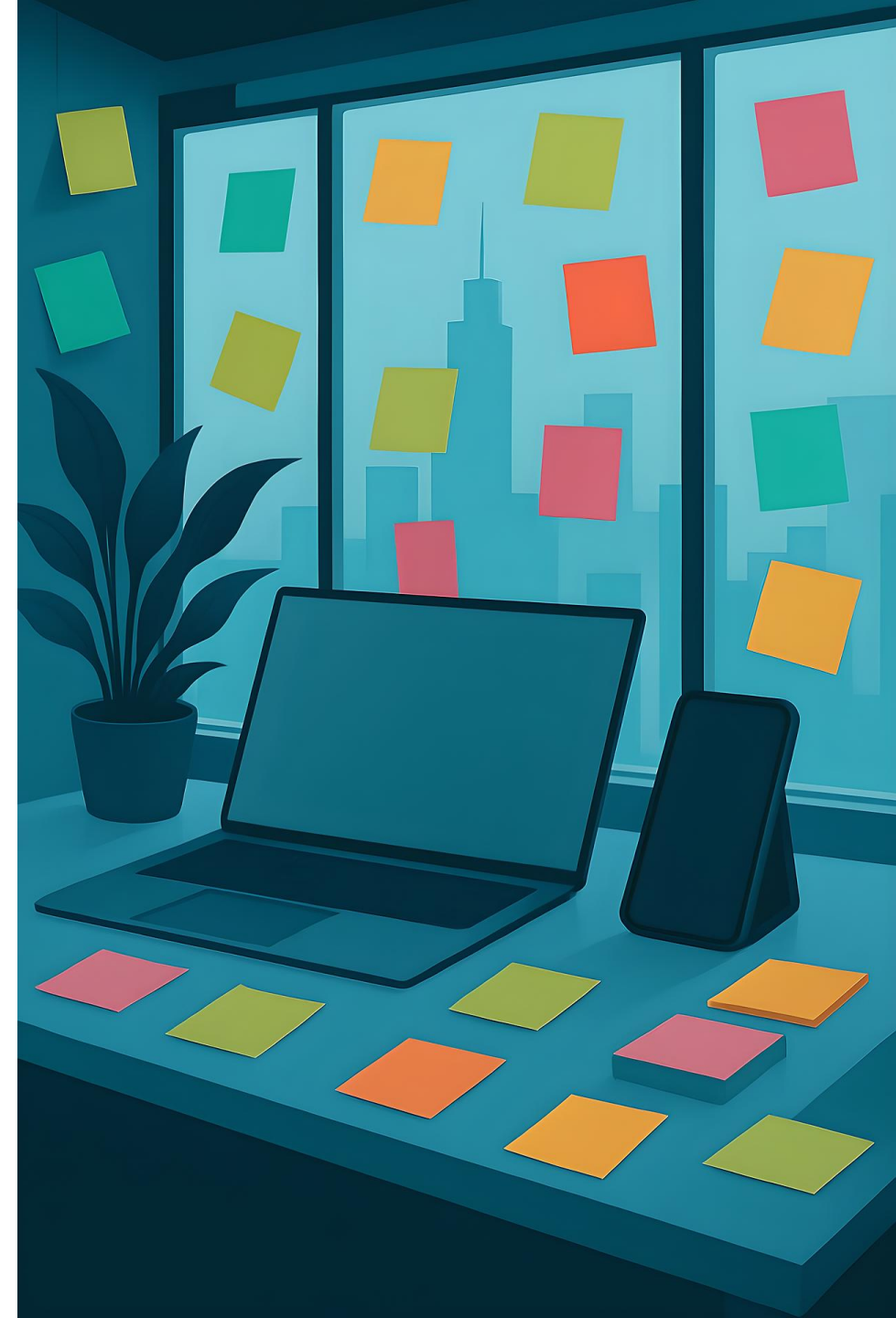
## **Programiranje za UNIX**

**Doc. dr. sc. Maja Braović**

23. listopada 2025.

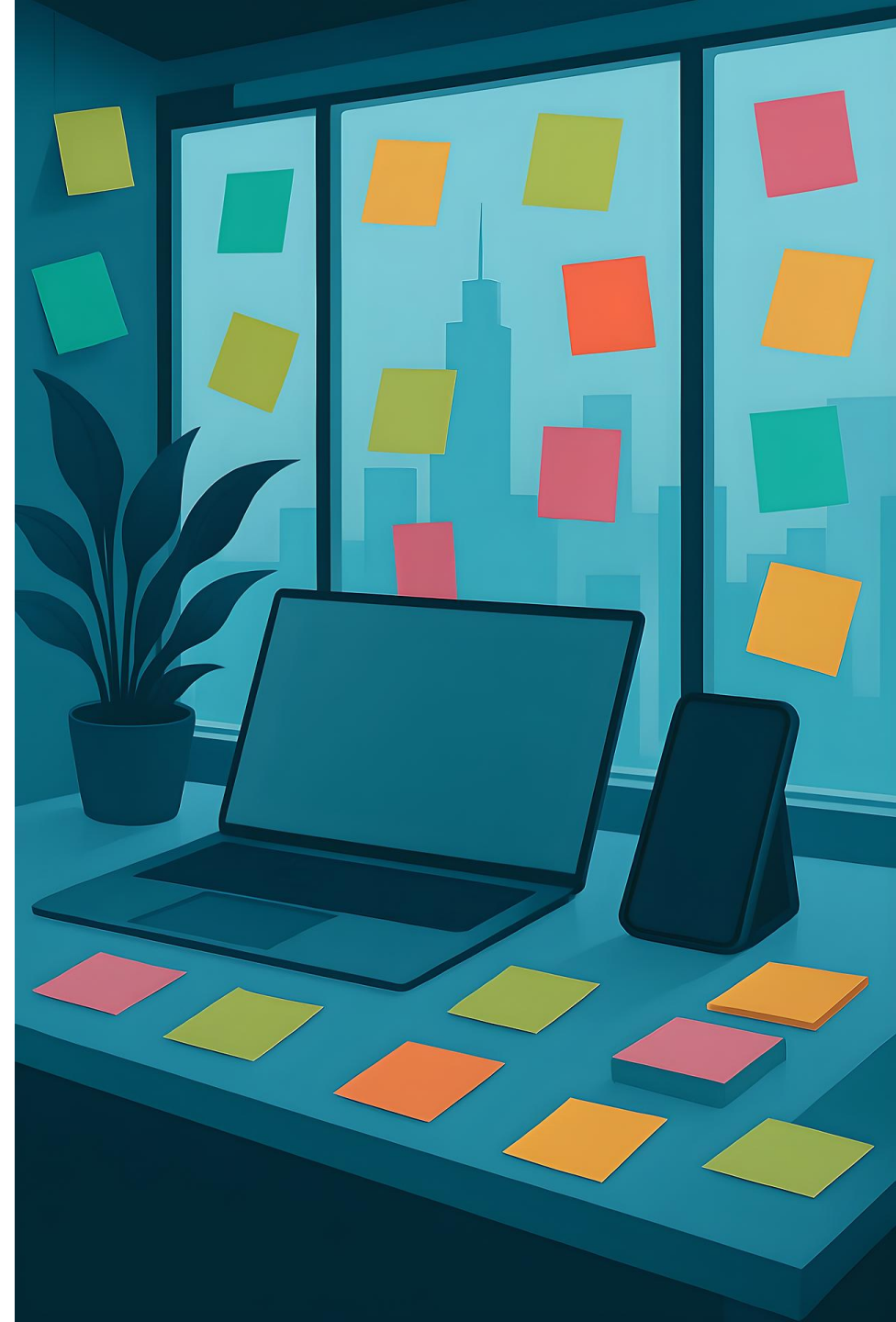
# Sadržaj

- Uvod
- Makefile datoteka
  - Rutina **make**
  - Struktura Makefile datoteke
  - Makefile varijable
  - Primjer Makefile datoteke
  - Rekurzivni Makefile
  - Primjer rekurzivne Makefile datoteke



# Sadržaj

- Standardizacija
  - ISO
  - IEC
  - ISO C (ISO/IEC 9899)
  - IEEE 1003.1 (POSIX)
- UNIX procesi
  - Memorijska slika UNIX procesa
- Literatura



# Rutina **make**

- Rutina **make** je alat koji automatizira proces kompajliranja i povezivanja programa prema pravilima definiranim u datoteci koja se najčešće (ukoliko nije drugačije navedeno) zove **Makefile**.
- Sintaksa: **make -f <datoteka> <opcije>**
- Primjer: **make -f moj\_makefile**
- Opcija „**-f**” u gornjim naredbama znači da rutina **make** ne traži zadani "Makefile" ili "makefile", nego koristi točno navedenu datoteku **moj\_makefile**.
- Glavna prednost **make** rutine je što izvršava samo one naredbe koje su potrebne, odnosno samo one dijelove koda koji su promijenjeni ili zastarjeli u odnosu na prethodne rezultate.

# Struktura **Makefile** datoteke

- Sintaksa: **target: dependencies**  
**commands**
- **target** – naziv pravila (ime programa) koje želimo pozvati
- **dependencies** – datoteke o kojima ovisi prevođenje
- **commands** – akcija koja se izvršava za generiranje datoteke
- Redak sa naredbama obavezno započinje sa <tab>.

# Struktura **Makefile** datoteke

```
vjezba37: vjezba.c
```

```
gcc vjezba.c -o vjezba37
```

# Struktura **Makefile** datoteke

Ovo želimo dobiti.

```
vjezba37: vjezba.c
```

```
gcc vjezba.c -o vjezba37
```

# Struktura **Makefile** datoteke

Ovo želimo dobiti.

vjezba37: vjezba.c

Ovo nam treba da to dobijemo.

gcc vjezba.c -o vjezba37



# Struktura **Makefile** datoteke

Ovo želimo dobiti.

vjezba37: vjezba.c

Ovo nam treba da to dobijemo.

gcc vjezba.c -o vjezba37

Ovo je naredba pomoću koje ćemo to ostvariti.

# Struktura **Makefile** datoteke

program: main.o funkcije.o

gcc main.o funkcije.o -o program

main.o: main.c

gcc -c main.c

funkcije.o: funkcije.c

gcc -c funkcije.c

# Struktura **Makefile** datoteke

```
program: main.o funkcije.o
```

```
gcc main.o funkcije.o -o program
```

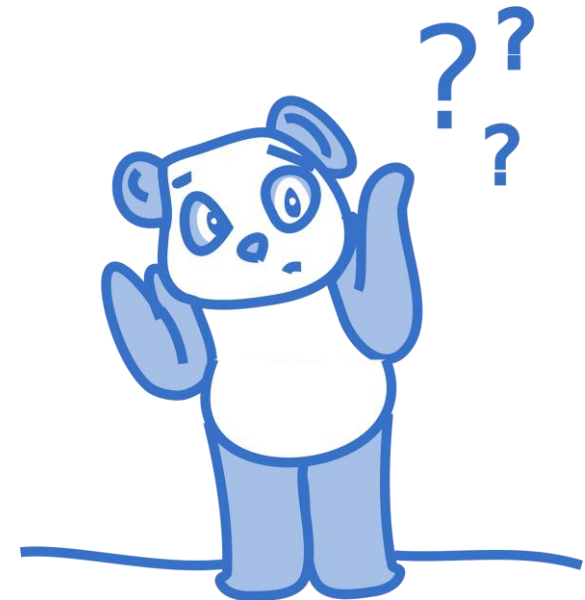
```
main.o: main.c
```

```
gcc -c main.c
```

```
funkcije.o: funkcije.c
```

```
gcc -c funkcije.c
```

Što mislite zašto ovdje nismo  
koristili main.c i funkcije.c?



Slika preuzeta iz [5].

# Struktura **Makefile** datoteke

```
program: main.o funkcije.o
```

```
gcc main.o funkcije.o -o program
```

```
main.o: main.c
```

```
gcc -c main.c
```

```
funkcije.o: funkcije.c
```

```
gcc -c funkcije.c
```

Što mislite zašto ovdje nismo koristili main.c i funkcije.c?

Da bismo izbjegli kompajliranje obje datoteke u slučaju da se promijeni samo jedna .c datoteka.

# Makefile variable

- Varijabla je simboličko ime definirano u Makefile datoteci, a predstavlja tekstualni string.

**FLAGS** = -03 -Wall

IME VARIABLE

VRIJEDNOST (STRING)

- Vrijednost varijable referencira se korištenjem oznake iza koje slijedi ime varijable u zagradama:

**\$ (FLAGS)**

# Primjer **Makefile** datoteke

```
CC = /usr/bin/gcc
CFLAGS = -Wall -O3
TARGETS = hello count
default: count
all: $(TARGETS)
```

```
hello: hello.o
    $(CC) $(CFLAGS) hello.o -o hello
```

```
count: count.o fn.o
    $(CC) $(CFLAGS) count.o fn.o -o count
```

```
clean :
    rm -f *.o *~ $(TARGETS)
```

```
.c.o:
    $(CC) $(CFLAGS) -c $<
```

# Primjer **Makefile** datoteke

**CC** = /usr/bin/gcc

CFLAGS = -Wall -O3

TARGETS = hello count

default: count

all: \$(TARGETS)

hello: hello.o

\$(CC) \$(CFLAGS) hello.o -o hello

count: count.o fn.o

\$(CC) \$(CFLAGS) count.o fn.o -o count

clean :

rm -f \*.o \*~ \$(TARGETS)

.c.o:

\$(CC) \$(CFLAGS) -c \$<

Definira varijablu **CC** koja predstavlja kompajler koji će se koristiti — ovdje je to **GCC** s putanjom **/usr/bin/gcc**.

# Primjer **Makefile** datoteke

CC = /usr/bin/gcc

**CFLAGS = -Wall -O3**

TARGETS = hello count

default: count

all: \$(TARGETS)

hello: hello.o

\$(CC) \$(CFLAGS) hello.o -o hello

count: count.o fn.o

\$(CC) \$(CFLAGS) count.o fn.o -o count

clean :

rm -f \*.o \*~ \$(TARGETS)

.c.o:

\$(CC) \$(CFLAGS) -c \$<

Definira opcije kompajlera koje će se dodavati kod svake kompilacije. „**-Wall**” uključuje sva standardna upozorenja, a „**-O3**” aktivira visoku razinu optimizacije koda.



# Primjer **Makefile** datoteke

CC = /usr/bin/gcc

CFLAGS = -Wall -O3

**TARGETS = hello count**

default: count

all: \$(TARGETS)

hello: hello.o

\$(CC) \$(CFLAGS) hello.o -o hello

count: count.o fn.o

\$(CC) \$(CFLAGS) count.o fn.o -o count

clean :

rm -f \*.o \*~ \$(TARGETS)

.c.o:

\$(CC) \$(CFLAGS) -c \$<

Popis glavnih izvršnih datoteka  
koje će biti generirane iz  
projekta: **hello** i **count**.

# Primjer **Makefile** datoteke

```
CC = /usr/bin/gcc
CFLAGS = -Wall -O3
TARGETS = hello count
default: count
all: $(TARGETS)
```

```
hello: hello.o
    $(CC) $(CFLAGS) hello.o -o hello

count: count.o fn.o
    $(CC) $(CFLAGS) count.o fn.o -o count

clean :
    rm -f *.o *~ $(TARGETS)

.c.o:
    $(CC) $(CFLAGS) -c $<
```

Glavni (zadani) cilj za **make** naredbu. Ako korisnik samo upiše **make**, kompajlirat će se cilj **count**.

# Primjer **Makefile** datoteke

```
CC = /usr/bin/gcc
CFLAGS = -Wall -O3
TARGETS = hello count
default: count
all: $(TARGETS)
```

```
hello: hello.o
    $(CC) $(CFLAGS) hello.o -o hello
```

```
count: count.o fn.o
    $(CC) $(CFLAGS) count.o fn.o -o count
```

```
clean :
    rm -f *.o *~ $(TARGETS)
```

```
.c.o:
    $(CC) $(CFLAGS) -c $<
```

Cilj **all** ovisi o svim ciljevima iz varijable **TARGETS** (**hello** i **count**). Pokretanjem **make all** kompajlirat će se obje izvršne datoteke.

# Primjer **Makefile** datoteke

```
CC = /usr/bin/gcc
CFLAGS = -Wall -O3
TARGETS = hello count
default: count
all: $(TARGETS)
```

## hello: hello.o

```
$(CC) $(CFLAGS) hello.o -o hello
```

```
count: count.o fn.o
```

```
$(CC) $(CFLAGS) count.o fn.o -o count
```

```
clean :
```

```
rm -f *.o *~ $(TARGETS)
```

```
.c.o:
```

```
$(CC) $(CFLAGS) -c $<
```

Cilj **hello** ovisi o objektnoj datoteci **hello.o**.

# Primjer **Makefile** datoteke

```
CC = /usr/bin/gcc
CFLAGS = -Wall -O3
TARGETS = hello count
default: count
all: $(TARGETS)
```

```
hello: hello.o
    $(CC) $(CFLAGS) hello.o -o hello
```

```
count: count.o fn.o
    $(CC) $(CFLAGS) count.o fn.o -o count
```

```
clean :
    rm -f *.o *~ $(TARGETS)
```

```
.c.o:
    $(CC) $(CFLAGS) -c $<
```

Ovaj redak sadrži naredbu za povezivanje **hello.o** u izvršnu datoteku **hello**, uz korištenje definiranog kompajlera i opcija.

# Primjer **Makefile** datoteke

```
CC = /usr/bin/gcc
CFLAGS = -Wall -O3
TARGETS = hello count
default: count
all: $(TARGETS)
```

```
hello: hello.o
    $(CC) $(CFLAGS) hello.o -o hello
```

```
count: count.o fn.o
    $(CC) $(CFLAGS) count.o fn.o -o count
```

```
clean :
    rm -f *.o *~ $(TARGETS)
```

```
.c.o:
    $(CC) $(CFLAGS) -c $<
```

Cilj **count** ovisi o objektnoj datoteci **count.o** i **fn.o**.

# Primjer **Makefile** datoteke

```
CC = /usr/bin/gcc
CFLAGS = -Wall -O3
TARGETS = hello count
default: count
all: $(TARGETS)
```

```
hello: hello.o
    $(CC) $(CFLAGS) hello.o -o hello
```

```
count: count.o fn.o
    $(CC) $(CFLAGS) count.o fn.o -o count
```

```
clean :
    rm -f *.o *~ $(TARGETS)
```

```
.c.o:
    $(CC) $(CFLAGS) -c $<
```

Ovaj redak sadrži naredbu za povezivanje **count.o** i **fn.o** u izvršnu datoteku **count**, uz korištenje definiranog kompajlera i opcija.

# Primjer **Makefile** datoteke

```
CC = /usr/bin/gcc
CFLAGS = -Wall -O3
TARGETS = hello count
default: count
all: $(TARGETS)
```

```
hello: hello.o
    $(CC) $(CFLAGS) hello.o -o hello
```

```
count: count.o fn.o
    $(CC) $(CFLAGS) count.o fn.o -o count
```

```
clean :
    rm -f *.o *~ $(TARGETS)
```

```
.c.o:
    $(CC) $(CFLAGS) -c $<
```

Cilj **clean** služi za čišćenje projekta od datoteka nastalih kompilacijom. Budući da nema ovisnosti, njegova se naredba može ručno pokrenuti sa **make clean**.



# Primjer **Makefile** datoteke

```
CC = /usr/bin/gcc
CFLAGS = -Wall -O3
TARGETS = hello count
default: count
all: $(TARGETS)
```

```
hello: hello.o
      $(CC) $(CFLAGS) hello.o -o hello
```

```
count: count.o fn.o
      $(CC) $(CFLAGS) count.o fn.o -o count
```

```
clean :
      rm -f *.o *~ $(TARGETS)
```

```
.c.o:
      $(CC) $(CFLAGS) -c $<
```

Ovaj redak uklanja sve objektne datoteke (**.o**), sve datoteke koje završavaju s **~** (privremene sigurnosne kopije) i sve izvršne datoteke navedene u varijabli **\$(TARGETS)** (o ovom slučaju **hello i count**).

“**-f**” znači “force”, tj. prisilno brisanje bez prikazivanja pogrešaka ako datoteka ne postoji.

# Primjer **Makefile** datoteke

```
CC = /usr/bin/gcc
CFLAGS = -Wall -O3
TARGETS = hello count
default: count
all: $(TARGETS)
```

```
hello: hello.o
    $(CC) $(CFLAGS) hello.o -o hello
```

```
count: count.o fn.o
    $(CC) $(CFLAGS) count.o fn.o -o count
```

```
clean :
    rm -f *.o *~ $(TARGETS)
```

```
.c.o:
    $(CC) $(CFLAGS) -c $<
```

Pravilo za pretvaranje izvornih C datoteka (.c) u objektne (.o) datoteke.

# Primjer **Makefile** datoteke

```
CC = /usr/bin/gcc
CFLAGS = -Wall -O3
TARGETS = hello count
default: count
all: $(TARGETS)

hello: hello.o
    $(CC) $(CFLAGS) hello.o -o hello

count: count.o fn.o
    $(CC) $(CFLAGS) count.o fn.o -o count

clean :
    rm -f *.o *~ $(TARGETS)

.c.o:
    $(CC) $(CFLAGS) -c $<
```

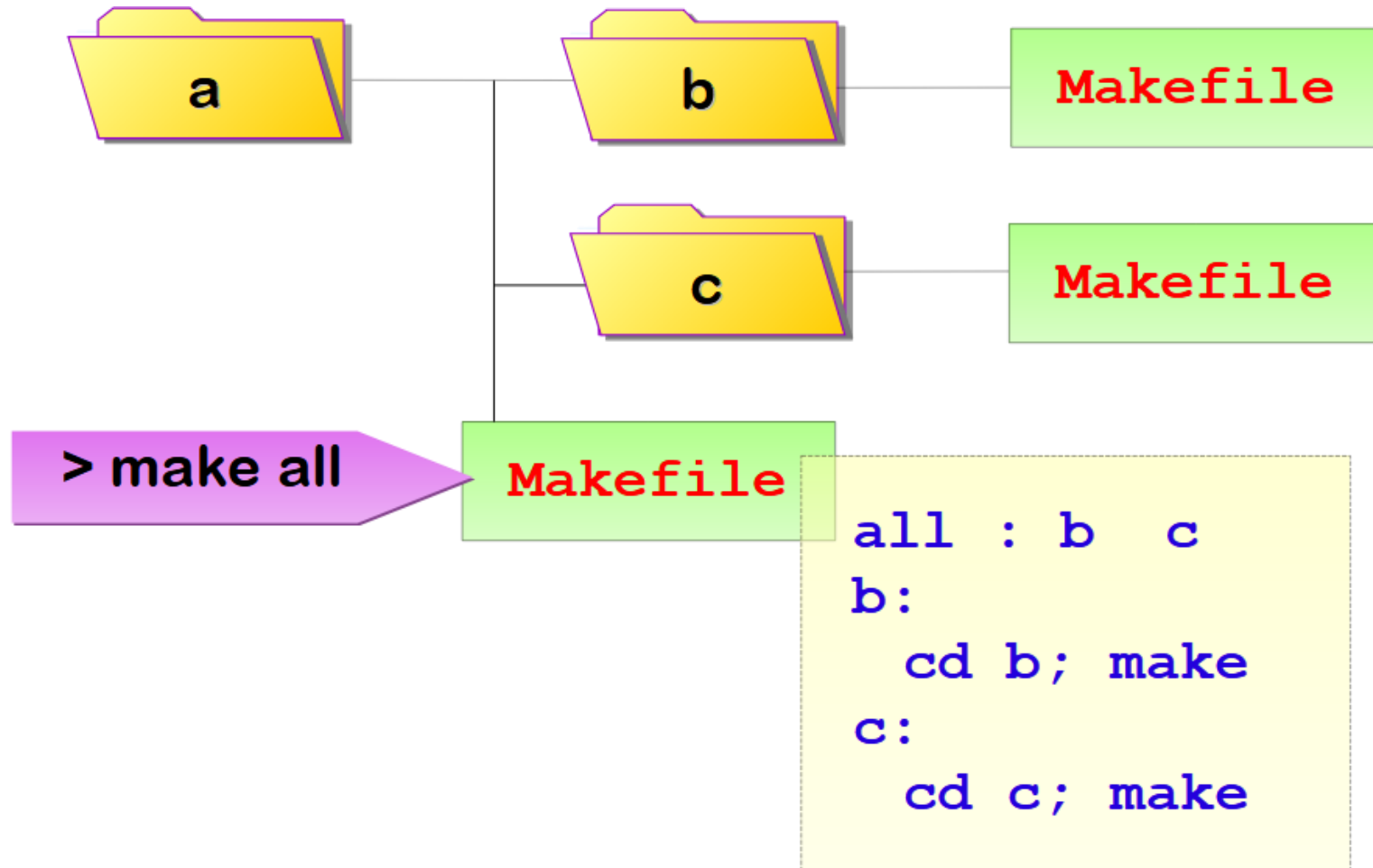
Ovom linijom se kompajlira  
ulazna **.c** datoteka u **.o**  
datoteku bez povezivanja.

“\$<” predstavlja prvu ovisnu  
datoteku, tj. ime **.c** datoteke  
koja se kompajlira.

# Rekurzivni **Makefile**

- Rekurzivni Makefile je poseban način organizacije Makefile-a koji se najčešće koristi kada imamo složenije projekte s podprojektima ili direktorijima.
- Rekurzija je u tome da se iz glavnog Makefile-a poziva **make** unutar poddirektorija, a ti poddirektoriji mogu sadržavati vlastite Makefile-ove s daljnjim pozivima **make**, i tako dalje.
- Ovim se omogućava modularna organizacija velikih projekata, gdje svaki podprojekt može biti neovisno kompajliran, održavan i testiran.
- Nedostatak ovakve organizacije je potencijalna složenost koja u određenim slučajevima uopće nije ni potrebna.

# Rekurzivni Makefile



# Primjer rekurzivne **Makefile** datoteke

```
SUBDIRS = projekt1 projekt2
```

```
all: $(SUBDIRS)
```

```
$(SUBDIRS):  
    cd $@ && make
```

```
clean:  
    for dir in $(SUBDIRS); do \  
        cd $$dir && make clean; \  
    done
```

# Primjer rekurzivne **Makefile** datoteke

**SUBDIRS = projekt1 projekt2**

all: \$(SUBDIRS)

\$(SUBDIRS):  
    cd \$@ && make

clean:  
    for dir in \$(SUBDIRS); do \  
        cd \$\$dir && make clean; \  
    done

Definiranje varijable **SUBDIRS**  
koja sadrži dva poddirektorija:  
**projekt1 i projekt2.**

# Primjer rekurzivne **Makefile** datoteke

```
SUBDIRS = projekt1 projekt2
```

```
all: $(SUBDIRS)
```

```
$(SUBDIRS):  
    cd $@ && make
```

```
clean:  
    for dir in $(SUBDIRS); do \  
        cd $$dir && make clean; \  
    done
```

Cilj **all** ovisi o svim direktorijima navedenima u **SUBDIRS**. Kada pozovemo **make all** ili samo **make**, pokreće se pravilo za svaki od tih direktorija.



# Primjer rekurzivne **Makefile** datoteke

```
SUBDIRS = projekt1 projekt2
```

```
all: $(SUBDIRS)
```

```
$(SUBDIRS):
```

```
    cd $@ && make
```

```
clean:
```

```
    for dir in $(SUBDIRS); do \
```

```
        cd $$dir && make clean; \
```

```
done
```

Definicija pravila za svaki  
direktorij u **SUBDIRS**.

# Primjer rekurzivne **Makefile** datoteke

```
SUBDIRS = projekt1 projekt2
```

```
all: $(SUBDIRS)
```

```
$(SUBDIRS):
```

```
    cd $@ && make
```

```
clean:
```

```
    for dir in $(SUBDIRS); do \  
        cd $$dir && make clean; \  
    done
```

Za svaki od direktorija u **SUBDIRS** (određenih sa \$@) prebacujemo se u taj direktorij (cd \$@) i izvršavamo naredbu **make** u njemu, što znači da će se pokrenuti tamošnji **Makefile** proces.

„\$@” predstavlja ime trenutnog cilja, tj. naziv direktorija (**projekt1** ili **projekt2**).

# Primjer rekurzivne **Makefile** datoteke

```
SUBDIRS = projekt1 projekt2
```

```
all: $(SUBDIRS)
```

```
$(SUBDIRS):
```

```
    cd $@ && make
```

**REKURZIJA**

```
clean:
```

```
    for dir in $(SUBDIRS); do \  
        cd $$dir && make clean; \  
    done
```

Za svaki od direktorija u **SUBDIRS** (određenih sa \$@) prebacujemo se u taj direktorij (cd \$@) i izvršavamo naredbu **make** u njemu, što znači da će se pokrenuti tamošnji **Makefile** proces.

„\$@” predstavlja ime trenutnog cilja, tj. naziv direktorija (**projekt1** ili **projekt2**).

# Primjer rekurzivne **Makefile** datoteke

```
SUBDIRS = projekt1 projekt2
```

```
all: $(SUBDIRS)
```

```
$(SUBDIRS):  
    cd $@ && make
```

**clean:**

```
    for dir in $(SUBDIRS); do \  
        cd $$dir && make clean; \  
    done
```

Definicija pravila **clean** koje se koristi za čišćenje (brisanje) generiranih datoteka u svim projektima.

# Primjer rekurzivne **Makefile** datoteke

```
SUBDIRS = projekt1 projekt2
```

```
all: $(SUBDIRS)
```

```
$(SUBDIRS):  
    cd $@ && make
```

```
clean:  
    for dir in $(SUBDIRS); do \  
        cd $$dir && make clean; \  
    done
```

Petlja koja prolazi kroz svaki direktorij u **SUBDIRS**.

„\  
” označava da se naredba nastavlja u sljedećem retku.

# Primjer rekurzivne **Makefile** datoteke

```
SUBDIRS = projekt1 projekt2
```

```
all: $(SUBDIRS)
```

```
$(SUBDIRS):
```

```
    cd $@ && make
```

```
clean:
```

```
    for dir in $(SUBDIRS); do \
```

```
        cd $$dir && make clean; \
```

```
    done
```

Ulazimo u svaki direktorij (**cd** **\$\$dir**), te pozivamo naredbu **make clean** u tom direktoriju.

„\” znak omogućava nastavak petlje.

„\$\$” označava doslovni znak „\$” koji će **shell** interpretirati, a ne Makefile.

# Primjer rekurzivne **Makefile** datoteke

```
SUBDIRS = projekt1 projekt2
```

```
all: $(SUBDIRS)
```

```
$(SUBDIRS):  
    cd $@ && make
```

```
clean:  
    for dir in $(SUBDIRS); do \  
        cd $$dir && make clean; \  
done
```

Kraj petlje.

# ISO

- ISO (**International Organization for Standardization**) je internacionalna organizacija za donošenje standarda koja je utemeljena 23. veljače 1947. godine.
- Trenutno sjedište ISO organizacije je u Ženevi u Švicarskoj.
- ISO organizacija trenutno ima 174 zemlje članice, a te zemlje imaju različite statusse članstva:
  - Puno (eng. *full*) članstvo uključuje pravo glasovanja i aktivno sudjelovanje u radu ISO tehničkih odbora.
  - Dopisno (eng. *correspondent*) članstvo ne uključuje pravo glasovanja, ali uključuje praćenje ISO rada.
  - Prijateljsko (eng. *subscriber*) članstvo uključuje zemlje s manjim kapacitetima koje prate rad ISO-a.
- **Hrvatska** je postala puna članica ISO organizacije 2005. godine, a u članstvu je predstavlja Hrvatski zavod za norme (HZN).





**ISO sjedište u Ženevi u Švicarskoj**

[https://www.iso.org/files/live/sites/isoorg/files/contact\\_ISO/information\\_for\\_visitors.pdf](https://www.iso.org/files/live/sites/isoorg/files/contact_ISO/information_for_visitors.pdf)

# What does ISO mean?

ISO is the short name for the International Organization for Standardization. It's not an acronym, but a name inspired by the Greek word isos, meaning "equal" – reflecting our mission to create standards that ensure consistency and equality worldwide. Because the organization's full name – and its initials – would vary across languages (for example Organisation internationale de normalisation in French), our founders chose "ISO" as a universal short form that could be recognized globally, regardless of language.

Whatever the country, whatever the language, we are always ISO.

<https://www.iso.org/about>

# IEC

- IEC (International Electrotechnical Commission) je međunarodna organizacija za normizaciju koja priprema i objavljuje međunarodne standarde za elektrotehniku, elektroniku i srodne tehnologije (elektrotehnologiju).
- Cilj IEC-a je osigurati sigurnost, pouzdanost i učinkovitost električnih sustava, proizvoda i tehnologija širom svijeta putem jedinstvenih standarda.
- IEC upravlja globalnim sustavima ocjene usklađenosti (eng. *conformity assessment*) koji certifikacijski potvrđuju usklađenost opreme, sustava i komponenti s međunarodnim normama.
- IEC trenutno ima 89 zemalja članica.
- **Hrvatska** je član IEC organizacije od 1993. godine.





### IEC sjedište u Ženevi u Švicarskoj

By IEC-CO - Own work, CC BY-SA 4.0, <https://commons.wikimedia.org/w/index.php?curid=55311118>

# ISO C (ISO/IEC 9899)

- ISO C je službeni standard koji definira kako se programski jezik C koristi i interpretira na globalnoj razini, što omogućava kompatibilnost i interoperabilnost među različitim platformama.
- ISO C standard pomaže u smanjenju razlika u implementaciji C jezika između različitih proizvođača kompajlera, čime se smanjuju problemi prilikom prijenosa koda.
- ISO C je definiran pod nazivom **ISO/IEC 9899**.
- Najnovija verzija standarda je ISO/IEC 9899:2024, koja specificira oblik i tumačenje programa napisanih u C-u.

# ISO C (ISO/IEC 9899)

- ISO C standard definira skup standardnih zaglavlja (eng. *header files*) koji obuhvaćaju različite funkcionalnosti:
  - ulaz/izlaz,
  - upravljanje memorijom,
  - manipulacija stringovima,
  - matematičke funkcije,
  - rad s datumom i vremenom,
  - i drugo.



Slika preuzeta iz [8].

# ISO C (ISO/IEC 9899)

- Ključne biblioteke (libovi):
  - `stdio.h` – funkcije za ulaz i izlaz (npr. `printf`, `scanf`)
  - `stdlib.h` – opće pomoćne funkcije (npr. alokacija memorije, konverzije)
  - `string.h` – rad sa nizovima i stringovima (npr. `strcpy`, `strlen`)
  - `limits.h` – definirane granice za tipove podataka
  - `errno.h` – upravljanje i provjera pogrešaka
  - `math.h` – matematičke funkcije (npr. `sin`, `cos`, `sqrt`)
  - `time.h` – rad s vremenom i datumima
  - `ctype.h` – funkcije za provjeru i manipulaciju znakovima

# IEEE 1003.1 (POSIX)

- **IEEE 1003.1** je standard koji definira sučelje operacijskog sustava kako bi se osigurala prenosivost aplikacija između različitih UNIX i UNIX sličnih sustava.
- **IEEE** – Institute of Electrical and Electronics Engineers.
- IEEE 1003.1 standard je poznat i pod nazivom **POSIX** (**P**ortable **O**perating **S**ystem **I**nterface (for UNIX)) standard.
- Standard uključuje osnovni ISO C jezik te nadograđuje funkcionalnosti uvodeći dodatna zaglavlja potrebna za rad s operativnim sustavom.
- Ukratko, IEEE 1003.1 je ključni standard koji propisuje kako programski kod treba komunicirati s operacijskim sustavom na način koji omogućuje kompatibilnost i prenosivost između različitih implementacija UNIX sustava i UNIX sličnih sustava (npr. macOS).



# IEEE 1003.1 (POSIX)

- Obavezna POSIX zaglavlja:
  - **unistd.h**
    - Sadrži definicije simboličkih konstanti i osnovnih funkcija za interakciju sa sustavom poput upravljanja datotekama i procesima.
    - Ime dolazi od „UNIX Standard Header”.
  - **sys/socket.h**
    - Definira API za rad s mrežnim socketima, što omogućava komunikaciju između računala preko mreže.
  - **sys/types.h**
    - Sadrži definicije osnovnih tipova podataka koje operacijski sustav koristi (npr. veličine i vrste podataka potrebne za rad s datotekama i procesima).

# IEEE 1003.1 (POSIX)

- Neka od opcionalnih POSIX zaglavlja:
  - **pthread.h**
    - Pruža sučelje za upravljanje nitima izvođenja (eng. *threads*), što omogućava paralelno izvršavanje koda u okviru iste aplikacije.
  - **sys/resource.h**
    - Upravlja resursima procesa, poput ograničenja upotrebe memorije ili CPU-a.
  - **sys/wait.h**
    - Funkcije za čekanje na završetak procesa.
    - U nekim verzijama POSIX standarda ovo zaglavlje može biti obavezno.

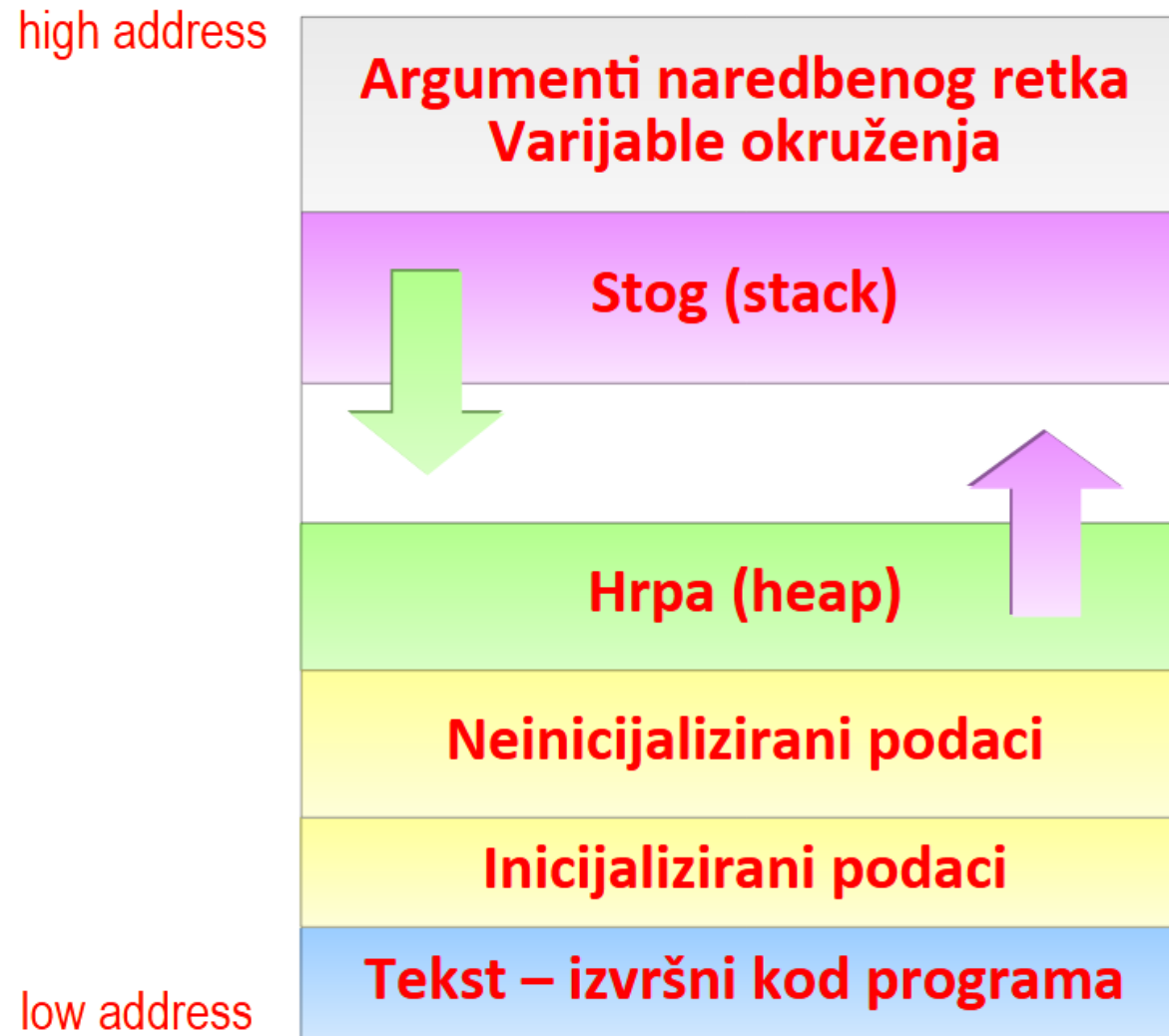
# UNIX procesi

- Proces u UNIX operacijskom sustavu je aktivni entitet koji nastaje izvođenjem programa.
- Svaki proces ima jedinstveni identifikator (PID).
- Moguće je postojanje više procesa istovremeno (eng. *multitasking*).
- Procesi imaju stanja poput „spremno za izvršavanje”, „u izvršavanju” i „blokirano”, a operacijski sustav prati njihove informacije u posebnoj tablici procesa.

# Memorijska slika UNIX procesa

- Memorijska slika UNIX procesa sastoji se od nekoliko ključnih dijelova, od kojih svaki ima posebnu ulogu u funkcioniranju procesa tijekom izvršavanja:
  - tekst,
  - inicijalizirani podaci,
  - neinicijalizirani podaci,
  - stog,
  - hrpa,
  - argumenti naredbenog retka i okruženje procesa.

# Memorijska slika UNIX procesa



# Memorijska slika UNIX procesa

- U memorijskoj slici UNIX procesa, "high address" i "low address" odnose se na najviše i najniže memorijske adrese koje proces koristi unutar svog virtualnog adresnog prostora.
- „**Low address**” (**nisko adresno područje**) obično je početak memorijskog prostora procesa, gde se često nalazi tekstualni segment (programski kod) i podaci.
- „**High address**” (**visoko adresno područje**) je kraj adresnog prostora.
- Rast stoga (eng. *stack*) i hrpe (eng. *heap*) kreće se u suprotnim smjerovima kako bi se izbjegao sudar u memorijskom prostoru.

# Tekst (strojni kod programa)

- **Tekst** (eng. *text segment*) je dio memorije u kojem se nalazi strojni kod programa.
- Ovaj segment služi samo za čitanje, što znači da program tijekom izvršenja ne može mijenjati svoj kod.
- Ovaj dio koda može biti dijeljen između više procesa koji koriste istu izvršnu datoteku, što štedi memorijske resurse.



```
if [[ !is_readable($temp_dir) ]] {  
    ('sys_get_temp_dir') { // sys_get  
    inaccessible temp dir, e.g. with  
};  
  
// see https://github.com/JamesHe  
redir');  
  
3.org/httpdocs/://tmp/"  
rray('/', '\\'), DIRECTORY_SEPARAT  
rray('/', '\\'), DIRECTORY_SEPARAT  
= DIRECTORY_SEPARATOR) {  
PARATOR;  
  
SEPARATOR, $open_basedir);  
redir) {  
    ) != DIRECTORY_SEPARATOR) {  
SEPARATOR;  
}
```

# Inicijalizirani podaci

- **Inicijalizirani podatkovni segment** (eng. *initialized data segment*) se učitava u memoriju prilikom pokretanja programa i postoji kroz cijelo trajanje izvođenja. Na primjer:
  - `int broj = 42;`
  - `char znak = 'A';`
  - `float konstanta = 3.14;`
- Ovdje se nalaze globalne i statičke varijable koje su inicijalizirane prije početka rada programa, tj. one imaju zadanu vrijednost definiranu u kodu.
- Ovaj segment se obično smješta u memoriju samo za čitanje, što može pomoći u zaštiti podataka od neželjenih izmjena tijekom izvođenja programa.

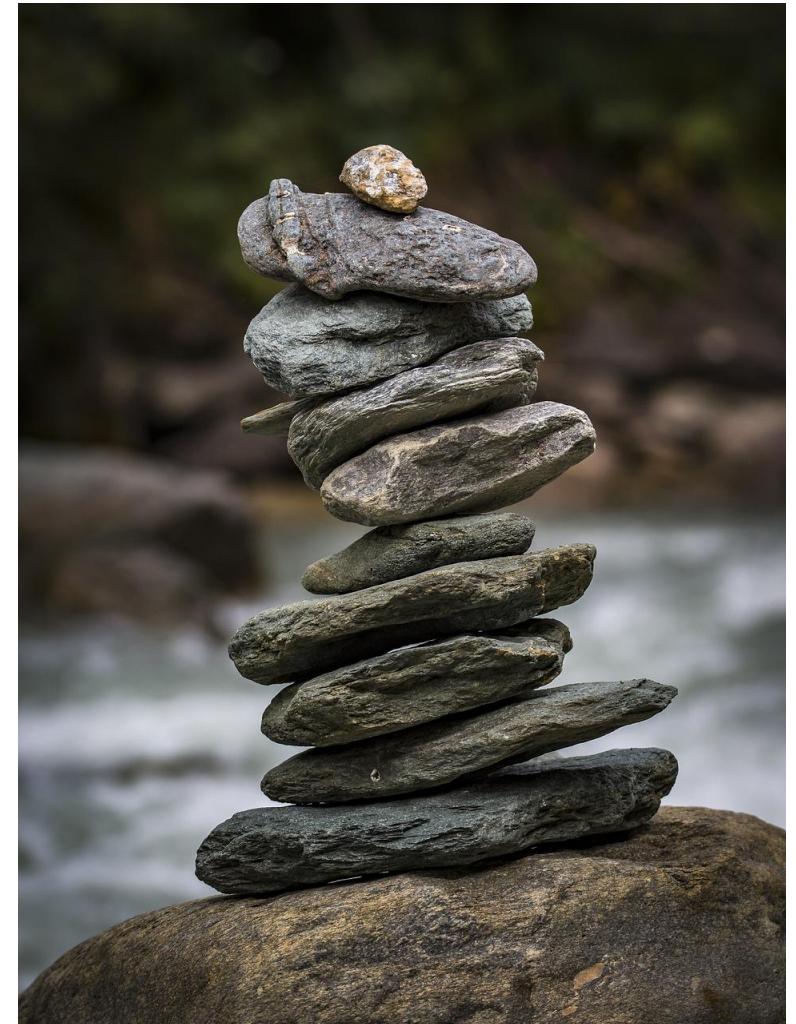


# Neinicijalizirani podaci

- **Neinicijalizirani podatkovni segment** (*eng. BSS segment – **Block Started by Symbol** segment*) sadrži globalne i statičke varijable koje nisu eksplicitno inicijalizirane u programu. Npr.:
  - `int globalVar;`
  - `static int staticVar;`
- Operacijski sustav ovakve varijable inicijalizira na nulu prije pokretanja procesa.
- Naziv „**BSS segment**” je povijesno ime koje datira iz ranih dana programskih jezika kada su blokovi za pohranu neinicijaliziranih podataka započinjali sa simbolom (adresom).
- Danas BSS blokovi ne počinju specifičnim simbolom, nego se za njihovu memorijsku rezervaciju koristi informacija o veličini segmenta (broj bajtova koji treba rezervirati) bez ikakvih spremljenih inicijaliziranih podataka.

# Stog (stack)

- **Stog** (eng. *stack*) je dinamički segment koji se koristi za pohranu okvira funkcijskih poziva.
- Funkcionira po principu **LIFO** (engl. *Last In, First Out*), što znači da je posljednji element koji je dodan na stog prvi koji se uklanja i obrađuje.
- Svaki put kada se pozove funkcija, na stog se pohranjuju informacije kao što su lokalne varijable, povratne adrese i parametri funkcije.
- Stog raste i smanjuje se tijekom izvršavanja funkcija.



Slika preuzeta iz [2].

# Hrpa (heap)

- **Hrpa** (eng. *heap*) je segment za **dinamičku alokaciju memorije** koju program može koristiti za spremanje podataka čija veličina i vrijeme postojanja nisu unaprijed poznati.
- Dinamička alokacija memorije odnosi se na memoriju alociranu putem funkcija poput **malloc**, **calloc**, **new**, itd.
- Veličina hrpe (heapa) može varirati tijekom izvođenja programa.
- Alokacija i oslobađanje memorije na hrpi kontrolira program ili alati za upravljanje memorijom.



Slika preuzeta iz [3].

# Argumenti naredbenog retka

- Prije samog pokretanja procesa, u memoriju se smještaju argumenti s kojima je proces pozvan (poput naziva i parametara) te varijable okruženja ili okoline (eng. *environment variables*) koje proces može koristiti za konfiguraciju svog rada.
- Argumenti naredbenog retka se primaju kao argumenti funkcije **main**:

```
int main(int argc, char *argv[])
```



**ARGUMENT COUNT**



**ARGUMENT VECTOR**



# Argumenti naredbenog retka

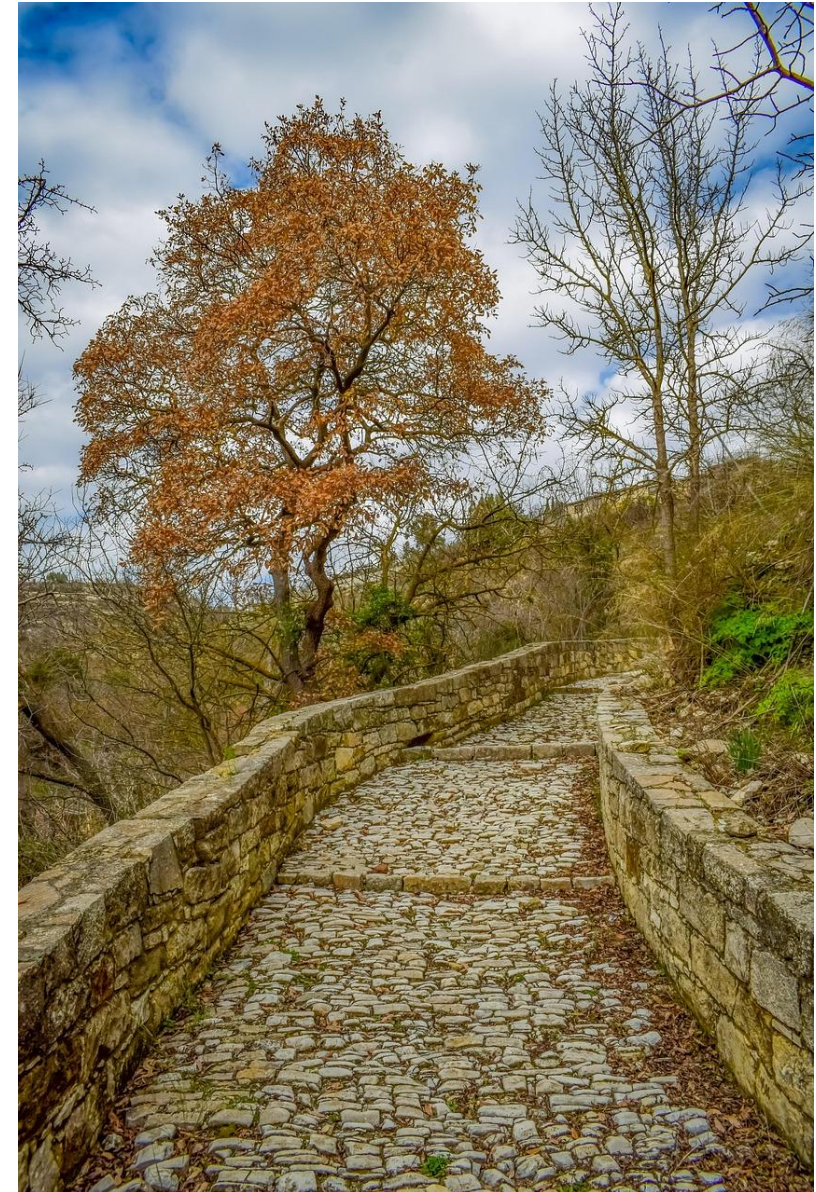
- **int argc** (**argument count**) označava broj argumenata koji su predani programu, uključujući i ime samog programa.
- **char \*argv[]** (**argument vector**) je niz pokazivača na znakove (stringove) koji sadrže stvarne argumente. Ime samog programa se označava sa argv[0], a ostali su argumenti koje korisnik unese putem naredbenog retka.
- Primjer: **./primjer prvi drugi treci**
  - argc = 4
  - argv[0] = "./primjer"
  - argv[1] = "prvi"
  - argv[2] = "drugi"
  - argv[3] = "treci"

# Variable okruženja

- Variable okruženja ili okoline (eng. *environment variables*) proces može koristiti za konfiguraciju svog rada.
- Pohranjene su u obliku parova **ključ-vrijednost** (ili **ime-vrijednost**).
- Varijabla okruženja **PATH** sadrži popis direktorija u kojima sustav traži izvršne datoteke.
  - `export PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin`
- Varijabla okruženja **HOME** pokazuje na početni direktorij korisnika.
  - `export HOME=/putanja/do/home`

# Varijable okruženja

- Varijable okruženja, ako nisu pravilno kontrolirane, mogu predstavljati sigurnosni rizik – na primjer, ako netko ubaci zlonamjernu putanju u **PATH**.
- Primjer:
  - PATH=**/home/user/zlonamjerni:/usr/bin:/bin**
  - Ako netko imenuje maliciozni program kao npr. **ls** i on se nalazi u **/home/user/zlonamjerni**, a korisnik unese naredbu **ls**, sustav će pokrenuti ovaj zlonamjerni program umjesto legitimnog **/bin/ls**.



Slika preuzeta iz [6].

# Varijable okruženja

- Povijesno gledano, koncept varijabli okruženja potječe još iz ranih UNIX verzija i predstavlja jedan od ključnih razloga zbog kojih su UNIX i UNIX slični sustavi toliko fleksibilni i moćni.
- Korištenje varijabli okruženja omogućilo je modularan i prilagodljiv način konfiguracije radnog okruženja i programa bez potrebe za izmjenom izvornog koda samih programa.



Slika preuzeta iz [7].



# Važni pojmovi

- Memorijska slika UNIX procesa sastoji se od:
  - segmenta teksta,
  - inicijaliziranih podataka,
  - neinicijaliziranih podataka,
  - stoga (eng. *stack*),
  - hrpe (eng. *heap*),
  - argumenata naredbenog retka i okruženja procesa.
- ISO C (ISO/IEC 9899) je službeni standard koji definira kako se programski jezik C koristi i interpretira na globalnoj razini, što omogućava kompatibilnost i interoperabilnost među različitim platformama.



Slika preuzeta iz [4].

# Literatura

- ISO – <https://www.iso.org/about>
- IEC – <https://www.iec.ch/homepage>
- IEEE – <https://www.ieee.org/>
- [1] <https://pixabay.com/photos/code-html-technology-programming-3622942/>
- [2] <https://pixabay.com/photos/stone-tower-balance-meditation-4519290/>
- [3] <https://pixabay.com/photos/hay-haystack-agriculture-nature-2571930/>
- [4] <https://pixabay.com/vectors/reminder-note-sticky-note-1922255/>
- [5] <https://pixabay.com/vectors/panda-confused-questions-shrug-303949/>
- [6] <https://pixabay.com/photos/path-cobblestone-cobbled-path-stone-4020610/>
- [7] <https://pixabay.com/vectors/penguin-animal-fedora-hat-linux-159524/>
- [8] <https://pixabay.com/vectors/development-web-design-graphic-4536630/>